

SyncFlow 产品需求说明书

1. 文档信息

项目	内容
项目名称	SyncFlow 数据同步与任务管理服务
文档类型	产品需求说明书（PRD）
当前版本	V1.0
后续版本	V1.1

这份文档只讲产品要做什么、为什么做，以及整体技术方向。具体的接口契约、字段规则、状态机、数据模型和页面细节，在对应的任务文档中发布，做到哪一部分就看哪一份文档。

2. 背景与问题

业务人员经常需要将 CSV 文件中的商品、订单或运营数据批量导入业务系统。人工录入效率低且容易遗漏；一次性脚本导入虽然快捷，但通常缺少任务状态、错误明细和重复导入保护。导入失败后，使用者很难判断是文件格式问题、某一行数据不合法，还是系统暂时不可用。

SyncFlow 提供一个可追踪的数据导入闭环：用户通过 React Web 页面提交 CSV 文件，系统创建任务并异步处理，校验每条记录，将合法数据写入 MySQL，并保存任务状态、处理统计、错误记录和执行日志。

3. 产品目标

3.1 用户目标

- 能够通过 Web 页面提交 CSV 文件并获得任务编号。
- 能够查看任务执行状态（等待中、执行中、成功、部分成功、失败、已取消等）。
- 能够查看处理统计（总行数、成功数、失败数）和失败原因。
- 能够安全地重复提交请求而不造成重复任务或重复数据。
- 在系统遇到暂时性错误时，能够知道任务是否正在重试及最终结果。

3.2 系统目标

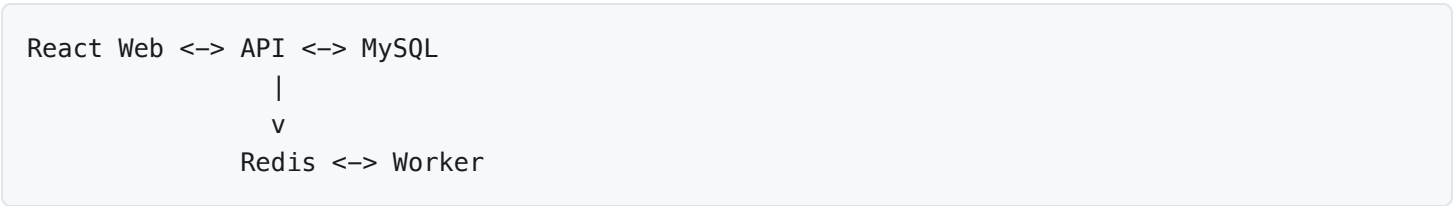
- 1. 任务创建接口要快速返回，别等整个文件处理完才响应。
- 2. 所有任务状态可追踪，符合状态转换规则。
- 3. 合法记录可靠写入 MySQL；错误记录可查询。
- 4. 同一任务内相同数据不重复写入。
- 5. React Web 客户端、服务、MySQL、Redis 和 Worker 能在一台本地电脑上通过 Docker Compose 启动。

4. 统一技术要求

后端实现可在两条语言路线中选择其一；除此之外，所有项目必须使用以下统一技术组件和工程约定。

能力	统一要求
后端语言与框架	Go + Gin, 或 Python + FastAPI, 二选一
Web 客户端	React 18+、TypeScript、Vite
前端路由	React Router
前后端通信	HTTP REST API；接口以 OpenAPI 为准
关系型数据库	MySQL 8.x
缓存与任务队列	Redis 7.x
容器与本地编排	Docker、Docker Compose
配置管理	.env 与 .env.example ；配置从环境变量读取
测试	后端单元测试与 MySQL 集成测试；前端关键交互测试
协作	Git、Issue、Pull Request、AI Code Review
持续集成	GitHub Actions 或等价 CI，执行格式化、检查、测试和构建

本地启动后的服务组成如下：



所有依赖必须在本地可运行；不得将核心功能建立在云服务、外部数据库或个人账号之上。

5. 用户角色

5.1 数据操作人员

负责提交 CSV 文件、创建同步任务、查询执行结果和查看错误明细。

5.2 系统维护人员

负责查看运行日志、定位失败任务、重试任务、维护本地运行环境和升级服务版本。

本项目不实现登录和权限控制，上述角色仅用于理解使用场景，所有功能对所有访问者开放。

6. 核心业务流程

准备 CSV 文件

-> 在 Web 页面选择文件并创建同步任务

-> 校验请求和文件

-> 在 MySQL 保存任务

-> 投递任务到 Redis

-> Worker 获取任务并执行

-> 逐行校验与写入 MySQL

-> 记录成功数、失败数和错误明细

-> 更新任务为最终状态

-> Web 页面展示任务、进度与错误记录

7. 功能概览

以下为产品的功能模块清单。各模块的详细需求（接口定义、字段规则、错误码、页面交互等）在对应的任务文档中发布，开发时以该文档为准。

编号	功能模块	说明
F1	创建同步任务	用户上传 CSV 文件创建异步导入任务
F2	任务查询	任务详情、任务列表（分页、状态筛选）
F3	CSV 校验与处理	文件级与行级校验、合法记录写入、错误记录保存
F4	异步 Worker	Redis 队列投递、Worker 消费、并发控制、优雅停止
F5	任务状态机	任务状态定义与合法转换
F6	重试机制	可重试/不可重试错误区分、退避策略、执行记录
F7	幂等与去重	幂等键、文件摘要、数据库唯一约束

编号	功能模块	说明
F8	任务取消	取消等待中或执行中的任务
F9	React Web 页面	任务列表、创建任务、任务详情、错误明细页面
F10	任务结果导出	任务完成后导出成功记录或错误明细为 CSV

8. 数据与接口原则

具体的表结构、字段定义、接口路径和错误码在周任务文档里按开发节奏发布。以下是总体原则，设计和实现时都要遵守。

1. 任务、成功记录、错误记录和执行尝试分开存储，方便查询和追溯。
2. 金额类字段用精确数值类型，别用浮点数。
3. 同一任务内的业务标识要唯一，防重复写入。
4. API 响应统一包裹成功数据和错误信息；错误带稳定的错误码和可读消息。
5. API 别返回内部文件路径、数据库连接信息或异常堆栈。
6. 列表类接口都要支持分页。
7. 上传文件存在受控目录，别直接拿客户端文件名当本地路径。

9. 非功能需求

9.1 本地可运行性

- `docker compose up` 应启动 Web、API、Worker、MySQL 和 Redis。
- `.env.example` 覆盖所有必需配置。
- README 包含启动、停止、迁移、前端构建、测试、查看日志和清理数据命令。
- 新开发者应在 15 分钟内完成基础启动。

9.2 前端工程质量

- 前端代码使用 TypeScript；页面、组件、API 客户端和领域类型应有清晰边界。
- 通用逻辑（状态展示、日期格式化、错误提示）应复用，避免在多个页面复制。
- 前端至少覆盖创建任务、任务列表和任务详情的关键交互测试。
- 生产构建必须成功，且不允许 TypeScript 类型错误。

9.3 安全性

- 不提交真实密码、密钥、数据库数据或 `.env` 文件。
- 上传文件保存在受控目录，文件名需规范化，禁止目录穿越。
- API 不返回内部文件路径、SQL、堆栈和连接配置。
- 日志不得记录密码、完整数据库 DSN 或敏感环境变量。

9.4 可观测性

- 所有日志至少包含时间、级别、服务名和消息。
- 与任务相关的日志必须包含任务 ID。
- 每次执行尝试必须可从日志和数据库记录中追溯。
- 提供健康检查接口用于进程存活检查。

9.5 性能基线

- 处理 1,000 行合法 CSV 时服务不应异常退出。
- 处理 10,000 行 CSV 时不得一次性无限制占用内存，应以流式读取或分批处理实现。
- 列表和错误明细接口必须分页。

10. 验收标准

最低要求 (V1.0)

1. Web、API、MySQL、Redis 和 Worker 均能在本地启动。
2. 能通过 Web 页面创建任务、异步处理 CSV、查询任务详情、任务列表和错误明细。
3. 能正确处理正常文件、空文件、表头错误、字段错误和重复业务标识。
4. 合法记录写入 MySQL，错误记录可查询。
5. 有数据库迁移、基础单元测试和 API 集成测试。
6. 所有变更通过分支、PR 和 AI Code Review 流程交付。
7. 前端具有任务列表、创建任务、任务详情和错误明细页面，并正确处理加载、空数据和请求失败状态。

V1.1 增强方向

1. 可配置的重试策略与执行记录。
2. 幂等键和文件摘要冲突检测。
3. 任务取消能力。

4. CI 自动执行格式化、静态检查、测试和构建。
5. 成功记录与错误明细 CSV 导出。
6. 性能测试或故障恢复测试报告。
7. 完整发布说明和复盘记录。

11. 研发协作要求

阅读 PRD

- > 提需求问题
- > 技术设计
- > 数据库与 API 设计
- > 分支开发
- > 本地测试
- > 提 PR
- > AI Code Review
- > 合并发布
- > 复盘